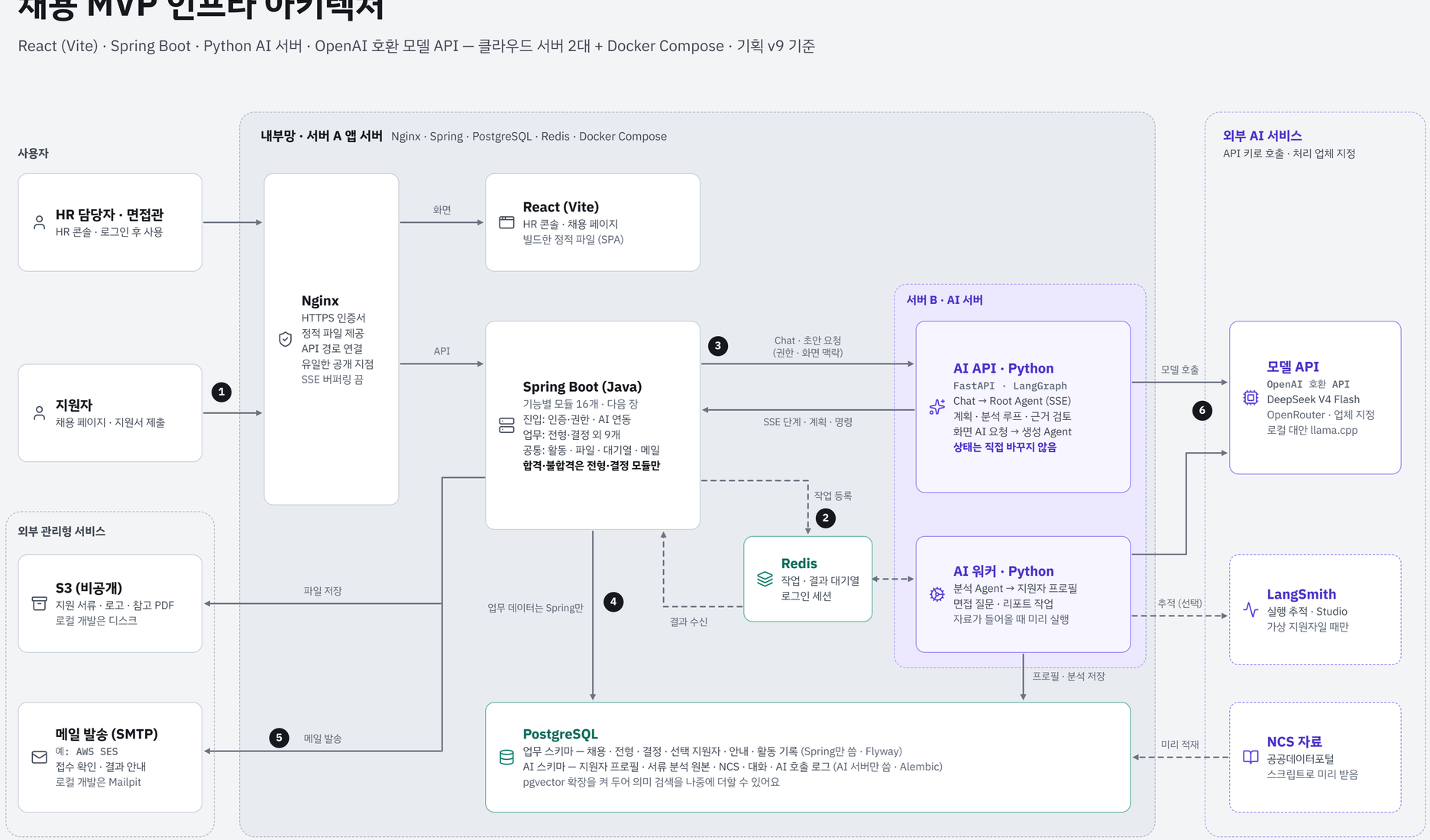
채용 MVP 인프라 아키텍처

React (Vite) · Spring Boot · Python AI 서버 · OpenAI 호환 모델 API — 클라우드 서버 2대 + Docker Compose · 기획 v9 기준



주요 흐름

1. **지원**
   지원자가 지원하면 Spring이 서류를 저장하고 지원번호가 담긴 접수 확인 메일을 보낸 뒤, 분석 작업을 Redis 대기열에 넣어요.
2. **미리 분석**
   AI 워커가 작업을 꺼내 분석 Agent를 실행하고 프로필과 서류 분석을 AI 스키마에 저장해요. 결과는 결과 스트림으로 돌려주고, Spring이 점수 · 경력 · 분석 상태를 기록해요.
3. **Chat 탐색**
   HR 질문 → Spring(권한 확인 · 화면 맥락) → Root Agent가 계획하고 처리기나 분석 루프로 실행한 뒤 근거를 검토해요. 단계 문구와 검증된 항목은 SSE로 먼저 보내요.
4. **선택과 결정**
   AI가 돌려준 넣기 · 빼기 · 면접 질문 명령은 Spring이 검증해 실행해요. 합격 · 불합격은 HR이 버튼으로 하고 활동 기록과 되돌리기가 남아요.
5. **안내 메일**
   결과 확정 뒤 그룹마다 Communication Agent 초안을 HR이 확인 · 수정하면, Spring이 변수와 전형별 안내를 채워 HTML 메일로 보내요.
6. **모델 호출**
   모델은 AI 서버의 LlmClient로만 불러요. 호출마다 AI 호출 로그를 남기고 요청 하나의 호출 수에 상한을 둬요. 지원서 분석은 배치로 모아 보낼 수 있어요.

서버 · 배포

서버 A 일반 VM (예: 4 vCPU · 16GB). Nginx · Spring · PostgreSQL · Redis

서버 B 일반 VM. AI API · AI 워커. GPU 없이 모델 API를 불러요.

모델 OpenAI 호환 API면 바꿔 쓸 수 있어요. 개발 · 평가는 DeepSeek V4 Flash. 운영 모델과 처리 업체는 개인정보 처리 위탁을 검토해 정해요.

배포 영역별 검사를 통과한 이미지를 두 서버에서 docker compose로 실행해요. DB 스키마는 Spring(Flyway)과 AI 서버(Alembic)가 따로 관리해요.

AI가 멈춰도

AI 서버나 모델 API가 멈춰도 공고 · 결정 · 확정 · 발송은 Spring만으로 계속돼요. 분석 작업은 대기열에 남았다가 다시 처리되고, 실패하면 분석 실패로 남아 HR이 다시 분석을 눌러요.

일부러 안 쓰는 것

쿠버네티스 · 마이크로서비스 · 별도 벡터 DB · Kafka · 자체 GPU 서빙. MVP 규모에선 운영 부담만 커요. 모듈만 잘 나눠 두면 나중에 분리할 수 있어요.

꼭 지킬 규칙

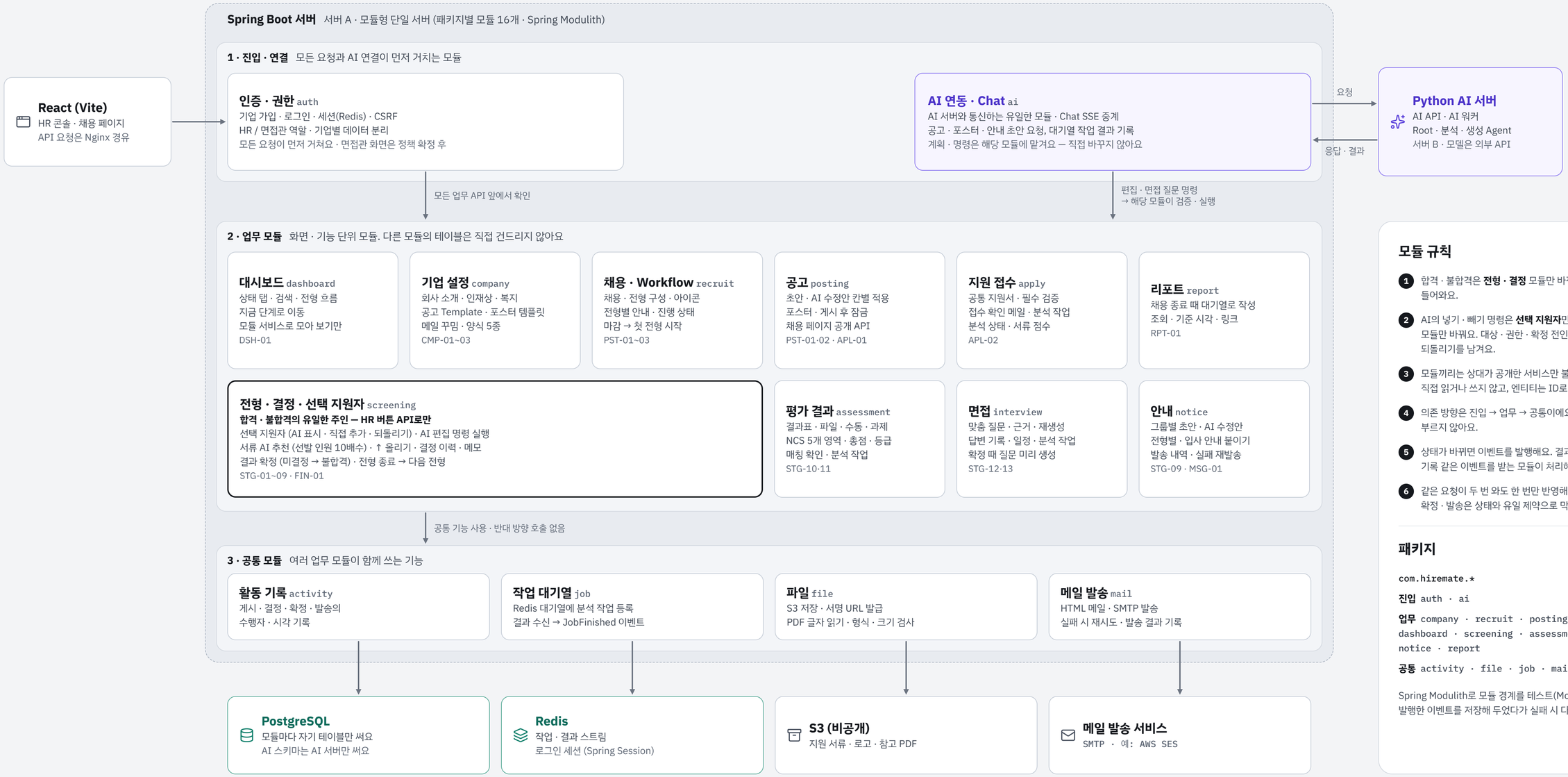
1. AI는 채용 결과를 바꾸지 않아요
AI 서버는 프로필 · 분석 결과 · 대화만 저장하고, 탐색 결과와 HR 지시는 '계획 · 명령'으로만 돌려줘요. 반영 · 이력 · 되돌리기는 Spring이, 합격 · 불합격은 HR 버튼으로만 해요.

2. 밖에서 열리는 건 Nginx뿐이에요
AI 서버 · DB · Redis는 내부망에만 둬요. 지원자 파일은 비공개 S3에 두고 서명 URL로만 열어요. 모델 API는 처리 업체를 지정해 부르고, 추적 도구는 가상 지원자일 때만 켜요.

3. 분석은 자료가 들어올 때, 기록은 처음부터
지원 · 평가 · 면접 자료가 들어오면 대기열로 미리 분석해요. 모델 호출은 AI 호출 로그에, 게시 · 결정 · 확정 · 발송은 활동 기록에 남겨요.

Spring Boot 기능별 모듈

서버 A의 Spring 서버 1개를 기능별 패키지(모듈)로 나눈 구조 — 경계를 테스트로 지키면 나중에 모듈 단위로 떼어낼 수 있어요



모듈 규칙

1. 합격 · 불합격은 **전형 · 결정** 모듈만 바꿔요. HR의 버튼 API로만 들어와요.
2. AI의 넣기 · 빼기 명령은 **선택 지원자**만, 면접 질문 명령은 **면접** 모듈만 바꿔요. 대상 · 권한 · 확정 전인지 검증하고 AI 표시와 되돌리기를 남겨요.
3. 모듈끼리는 상대가 공개한 서비스만 불러요. 다른 모듈의 테이블은 직접 읽거나 쓰지 않고, 엔티티는 ID로만 참조해요.
4. 의존 방향은 진입 → 업무 → 공통이에요. 공통 모듈은 업무 모듈을 부르지 않아요.
5. 상태가 바뀌면 이벤트를 발행해요. 결과 확정 · 작업 완료 · 면접 답변 기록 같은 이벤트를 받는 모듈이 처리해요.
6. 같은 요청이 두 번 와도 한 번만 반영해요. 같은 이메일 지원 · 결정 · 확정 · 발송은 상태와 유일 제약으로 막아요.

패키지

`com.hiremate.*`

진입 `auth · ai`

업무 `company · recruit · posting · apply · dashboard · screening · assessment · interview · notice · report`

공통 `activity · file · job · mail`

Spring Modulith로 모듈 경계를 테스트(ModularityTests)로 검사하고, 발행한 이벤트를 저장해 두었다가 실패 시 다시 보내요.

AI Agent 구조

실행 시점 3가지 · Root Agent + 하위 Agent 6개 (Interview Agent는 분석 · 생성 두 역할) · 다음 단계를 아는 일은 처리기로, 자료를 읽어 봐야 아는 일만 분석 루프로 — 기획 v9 §11~13

Root Agent · 분석 Agent · 생성 Agent
코드 · 도구 (LLM 없음) · 데이터 · 코드 LLM 처리 단계

① 자료가 들어올 때 · AI 워커가 분석 Agent를 미리 실행 (비동기)

자료 도착
지원 접수
평가 결과 등록
면접 답변 기록
Spring이 작업 등록

→

AI 워커
대기열에서 꺼내
작업 유형별 Agent 실행
결과는 결과 스트림으로

→

분석 Agent · Chat에서는 저장된 분석이 없거나 다시 검토할 때 실시간으로

LLM ① 사실 · 본인 역할 추출 → LLM ② 기준별 판정 → 코드 ③ 근거 원문 대조 → ↻ 확인 필요만 다시 판정 → 코드 ④ 점수 계산 · 저장

Application Agent
- 시점 지원 접수 때 (배치로 모아 보낼 수 있음)
- 입력 이력서 · 자기소개서 · 포트폴리오 · 자격증
  공고 평가 기준 (자격요건 · 우대사항)
- 결과 사실 · 판정 + 근거 · 문항별 요약
  서류 점수 자격 50 + 우대 50 · 보류

Assessment Agent
- 시점 평가 결과 등록 때
- 입력 NCS 5개 영역 점수 · 총점 · 등급
  결과 파일 · 과제 · URL · 평가 메모
- 결과 점수 · 등급은 모델에 보이지 않고 그대로
  강점 · 약점 + 근거

Interview Agent 결과 분석
- 시점 질문별 답변 기록 때
- 입력 맞춤 질문 · 답변 기록
  이전 면접의 확인할 점
- 결과 확인된 점 · 확인되지 않은 점
  답변 인용 + 근거 · 점수는 만들지 않음

→

지원자 프로필 DB
지원자별로 전형마다 쌓임
항목마다 근거 · 출처 · 확인 상태
점수 · 경력은 Spring에도
`PostgreSQL · ai 스키마`

② AI Chat 질문 · Root Agent가 계획하고, 처리기나 분석 루프로 실행한 뒤 근거를 검토

AI Chat 질문
현재 채용 · 전형 · 진행 상태
@멘션 · 체크 · 앞 대화 결과
자연어 질문
Spring이 권한 확인 후 전달

→

Root Agent
Recruitment Orchestrator
① **차단 확인**
원 요청의 차별 조건
② **계획 (모델 1회)**
요청 종류 · 대상 범위(scope)
③ **처리**
정해진 처리기 또는 분석 루프
④ **근거 검토 · 기록**
검증된 항목만 SSE · 대화방

계획 종류
`find · answer · questions · edit conversation · clarify · blocked · decision`

원문을 직접 해석하지 않음
채용 상태를 바꾸지 않음

→

찾기 · find
"~ 찾아줘" → 교체 · "~도" → 추가
"그중 ~" → 좁히기
"이 중 ~ 빼줘" → 빼기

→

검색 · 점수 필터 코드
경력 · 점수 · 영역 점수 · 등급
공고 기준 · 프로필 항목 검색
같은 입력엔 같은 후보와 순서

→

후보 근거 재확인 LLM 후보 · 병렬
저장된 프로필 · 서류 판정
근거가 조건을 받치는지 판정 · 서류는 AI 추천 안에서만

→

선택 지원자 반영
리스트 보라 테두리 · Chat 요약
Spring이 저장 · AI 표시

분석 · answer
지원자 의견 · 비교 · 현황
확인할 점이 필요한 면접 질문

→

분석 루프 LLM
미리 넣은 프로필에서 시작해
도구를 골라 다음 자료를 정해요
근거마다 번호 (E1, E2 ...)

→

조회 도구 · 하위 Agent 필요할 때만
프로필 · 서류 원본 · 검색 · 전형 · 회사
Application · 재검토 · Interview · 분석

→

근거 검토 → 답변
상태는 근거가 받치는 만큼만
과장은 확인 필요 · 무관은 버림
최종 답과 검토 호출을 남겨 둠

편집 · 질문 · 대화
넣기 · 빼기 · 비우기 · 되돌리기
면접 질문 · 대화 · 되묻기 · 차단
정해진 처리기

{ } 계획을 명령(JSON)으로만 바꿈 대상은 코드가 펼침
모델은 범위(멘션 · 체크 · 이전 결과 · 이름)만 말하고 지원자 목록은 코드가 정해요.
없는 이름 · 동명이인이면 되묻고, 이미 결정했거나 추천 밖인 지원자는 이유와 함께 빼요.
합격 · 불합격 요청은 실행하지 않고 '체크 후 맨 아래 버튼'으로 안내해요.

→

Spring 검증 · 실행
선택 지원자 · 면접 질문 반영
처리한 이름 알림

③ 화면 AI 요청 · 채용 이벤트 · Root Agent를 거치지 않고 생성 Agent를 실행

화면 · 이벤트
공고 · 포스터 AI 요청
결과 확정 (다음 면접)
안내 초안 · 수정 요청
채용 종료

→

생성 Agent · 초안만 만들고 게시 · 발송하지 않음 만들고 → 코드로 검사 → 필요하면 다시 씀

Posting Agent
- 시점 공고 · 포스터 화면의 AI 요청
  Template 채우기는 코드
- 입력 HR 요청 · 고른 줄 · 현재 공고
  작성 스타일 · NCS 능력단위
- 결과 바뀐 칸만 수정안 · NCS 근거 요건
  전형 이름 + 종류 · 필수 서류
  포스터 디자인 · 애매하면 되물음

Interview Agent 질문 생성
- 시점 결과 확정 때 (대기열)
  Chat 요청 때
- 입력 서류 판정 · 프로필 · 이전 면접
  인재상 · Root가 정한 확인할 점
- 결과 1차 자기소개 + 3 · 최종 + 5
  질문마다 유형 · 근거 · 확인할 내용

Communication Agent
- 시점 안내 초안 화면 · 그룹마다
- 입력 확정 결과 · 양식 4종 · 전형별 안내
- 결과 AI 작성 영역만 채운 초안
  필수 문구 · 변수 자리는 고정
  없는 평가 · 일정은 넣지 않음

Report Agent
- 시점 채용 종료 때 · 대기열
- 입력 전형별 집계 · 결정 · 메모
  서류 분석 · 지원자 프로필
- 결과 문서형 최종 리포트 · 기준 시각
  숫자는 집계값 그대로
  약점 · 미확인은 특이사항으로

→

초안 · 질문 · 리포트
HR이 확인 · 수정 · 적용한 뒤
게시 · 발송
AI는 게시 · 발송하지 않음

공통 기반 · 모든 Agent가 함께 씀

지원자 프로필
전형마다 쌓이는 정리 정보 · 정리한 Agent
확인됨 · 근거 있음 · 미확인 · 상충

검색 도구
조건 — 경력 · 점수 · 영역 점수 · 등급 · 결정
프로필 항목 검색 · 미확인 · 상충은 근거 제외

근거와 출처
원문에 없는 근거 문장은 버림
Root는 번호로 인용 · 코드가 원문 · 출처를 붙임

범위 · 호출 예산
회사 · 채용 · 전형 범위는 코드가 context로
지원서 속 지시문은 데이터로만 · 요청별 호출 예산

차별 조건 차단
나이 · 성별 · 출신 지역 · 혼인 여부 · 신체 조건
요청 · 검색 조건 · 면접 질문에 같은 검사

한 모델 · 역할별 지시문
Agent마다 모델을 따로 띄우지 않음
지시문 · 도구 · 출력 형식만 다르게 · NCS 공용

상태 변경은 Spring만
AI는 계획 · 명령 · 초안까지
서버가 검증 · 실행 · 이력 기록